

HDOC Day-16

Menu-driven program using switch-case and separate void user-defined functions

1. Problem Statement

Write a menu-driven C program using switch-case to perform digit-based operations on a positive integer. Define one separate user-defined function for each menu option, with void return type and a simple function call. Each function reads the required input, processes it, and displays the result.

- Check whether the digits are in increasing order from left to right.
- Check whether the digits are in decreasing order from left to right.
- Find the largest digit and the smallest digit.
- Find the second largest digit and second smallest digit in the number.

2. Algorithm

1. Start.
2. Display four menu options.
3. Read the user's choice.
4. Use switch-case to call the corresponding void function.
5. Inside the selected function, read a positive integer and process its digits.
6. Display the required result.
7. For an invalid choice, display an error message.
8. Stop.

Function-wise Logic

Choice 1 – Increasing order: Store the digits in an array in left-to-right order. Compare every digit with the next digit. If any digit is greater than or equal to the next digit, the digits are not in strictly increasing order.

Choice 2 – Decreasing order: Store the digits in left-to-right order. If any digit is less than or equal to the next digit, the digits are not in strictly decreasing order.

Choice 3 – Largest and smallest: Extract each digit using % 10. Update largest and smallest while digits remain.

Choice 4 – Second largest and second smallest: Mark which digits 0–9 occur. Scan upward for the smallest and second smallest distinct digits, and downward for the largest and second largest distinct digits.

3. Test Case Table

The table below covers both successful and unsuccessful cases for the four menu options.

Test	Choice	Input	Purpose	Expected Output
TC1	1	12345	Digits strictly increase	Digits are in increasing order
TC2	1	12435	Increasing-order failure	Digits are not in increasing order
TC3	2	97531	Digits strictly decrease	Digits are in decreasing order
TC4	2	97351	Decreasing-order failure	Digits are not in decreasing order
TC5	3	58320	Find extreme digits	Largest digit = 8; Smallest digit = 0
TC6	4	58320	Find second distinct extremes	Second largest digit = 5; Second smallest digit = 2
TC7	4	7777	No two distinct digits	Second largest/smallest digit does not exist

4. C Program

```
#include <stdio.h>

void increasingOrder(void);
void decreasingOrder(void);
void largestSmallest(void);
void secondLargestSmallest(void);

int main(void)
{
    int choice;

    printf("1. Check increasing order\n");
    printf("2. Check decreasing order\n");
    printf("3. Find largest and smallest digit\n");
    printf("4. Find second largest and second smallest digit\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1: increasingOrder(); break;
        case 2: decreasingOrder(); break;
        case 3: largestSmallest(); break;
        case 4: secondLargestSmallest(); break;
        default: printf("Invalid choice\n");
    }
    return 0;
}

void increasingOrder(void)
{
    long long n, temp;
    int a[20], count = 0, i, flag = 1;
    printf("Enter a positive integer: ");
    scanf("%lld", &n);
    temp = n;
    do { a[count++] = temp % 10; temp /= 10; } while (temp > 0);
    for (i = count - 1; i > 0; i--)
        if (a[i] >= a[i-1]) { flag = 0; break; }
    if (flag) printf("Digits are in increasing order\n");
    else printf("Digits are not in increasing order\n");
}

void decreasingOrder(void)
{
    long long n, temp;
    int a[20], count = 0, i, flag = 1;
    printf("Enter a positive integer: ");
```

```

scanf("%lld", &n);
temp = n;
do { a[count++] = temp % 10; temp /= 10; } while (temp > 0);
for (i = count - 1; i > 0; i--)
    if (a[i] <= a[i-1]) { flag = 0; break; }
if (flag) printf("Digits are in decreasing order\n");
else printf("Digits are not in decreasing order\n");
}

void largestSmallest(void)
{
    long long n, temp;
    int d, largest = 0, smallest = 9;
    printf("Enter a positive integer: ");
    scanf("%lld", &n);
    temp = n;
    do {
        d = temp % 10;
        if (d > largest) largest = d;
        if (d < smallest) smallest = d;
        temp /= 10;
    } while (temp > 0);
    printf("Largest digit = %d\n", largest);
    printf("Smallest digit = %d\n", smallest);
}

void secondLargestSmallest(void)
{
    long long n, temp;
    int present[10] = {0}, d, i;
    int smallest = -1, secondSmallest = -1;
    int largest = -1, secondLargest = -1;

    printf("Enter a positive integer: ");
    scanf("%lld", &n);
    temp = n;
    do { present[temp % 10] = 1; temp /= 10; } while (temp > 0);

    for (i = 0; i <= 9; i++)
        if (present[i]) {
            if (smallest == -1) smallest = i;
            else { secondSmallest = i; break; }
        }
    for (i = 9; i >= 0; i--)
        if (present[i]) {
            if (largest == -1) largest = i;
            else { secondLargest = i; break; }
        }

    if (secondLargest == -1)
        printf("Second largest digit does not exist\n");
    else
        printf("Second largest digit = %d\n", secondLargest);

    if (secondSmallest == -1)
        printf("Second smallest digit does not exist\n");
    else
        printf("Second smallest digit = %d\n", secondSmallest);
}

```

5. Sample Executions

Choice 1

Input: choice = 1, number = 12345

Output: Digits are in increasing order

Choice 2

Input: choice = 2, number = 97531

Output: Digits are in decreasing order

Choice 3

Input: choice = 3, number = 58320

Output: Largest digit = 8; Smallest digit = 0

Choice 4

Input: choice = 4, number = 58320

Output: Second largest digit = 5; Second smallest digit = 2

6. Screenshot Space

Paste clear screenshots after compiling and executing the program. Suggested screenshots are provided below.

Screenshot 1 – Source code and successful compilation

```
#include <stdio.h>
void increasingOrder(void);
void decreasingOrder(void);
void largestSmallest(void);
void secondLargestSmallest(void);
int main(void)
{
    int choice;
    printf("1. Check increasing order\n");
    printf("2. Check decreasing order\n");
    printf("3. Find largest and smallest digit\n");
    printf("4. Find second largest and second smallest digit\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);
    switch (choice)
    {
        case 1: increasingOrder(); break;
        case 2: decreasingOrder(); break;
        case 3: largestSmallest(); break;
        case 4: secondLargestSmallest(); break;
        default: printf("Invalid choice\n");
    }
    return 0;
}
void increasingOrder(void)
{
    long long n, temp;
    int a[20], count = 0, i, flag = 1;
```

```

int a[20], count = 0, i, flag = 1;
printf("Enter a positive integer: ");
scanf("%lld", &n);
temp = n;
do { a[count++] = temp % 10; temp /= 10; } while (temp > 0);
for (i = count - 1; i > 0; i--)
if (a[i] >= a[i-1]) { flag = 0; break; }
if (flag) printf("Digits are in increasing order\n");
else printf("Digits are not in increasing order\n");
}
void decreasingOrder(void)
{
long long n, temp;
int a[20], count = 0, i, flag = 1;
printf("Enter a positive integer: ");
scanf("%lld", &n);
temp = n;
do { a[count++] = temp % 10; temp /= 10; } while (temp > 0);
for (i = count - 1; i > 0; i--)
if (a[i] <= a[i-1]) { flag = 0; break; }
if (flag) printf("Digits are in decreasing order\n");
else printf("Digits are not in decreasing order\n");
}
void largestSmallest(void)
{
long long n, temp;
int d, largest = 0, smallest = 9;
printf("Enter a positive integer: ");

```

```

printf("Enter a positive integer: ");
scanf("%lld", &n);
temp = n;
do {
d = temp % 10;
if (d > largest) largest = d;
if (d < smallest) smallest = d;
temp /= 10;
} while (temp > 0);
printf("Largest digit = %d\n", largest);
printf("Smallest digit = %d\n", smallest);
}
void secondLargestSmallest(void)
{
long long n, temp;
int present[10] = {0}, d, i;
int smallest = -1, secondSmallest = -1;
int largest = -1, secondLargest = -1;
printf("Enter a positive integer: ");
scanf("%lld", &n);
temp = n;
do { present[temp % 10] = 1; temp /= 10; } while (temp > 0);
for (i = 0; i <= 9; i++)
if (present[i]) {
if (smallest == -1) smallest = i;
else { secondSmallest = i; break; }
}
for (i = 9; i >= 0; i--)

```

```

}
for (i = 9; i >= 0; i--)
if (present[i]) {
if (largest == -1) largest = i;
else { secondLargest = i; break; }
}
if (secondLargest == -1)
printf("Second largest digit does not exist\n");
else
printf("Second largest digit = %d\n", secondLargest);
if (secondSmallest == -1)
printf("Second smallest digit does not exist\n");
else
printf("Second smallest digit = %d\n", secondSmallest);
}
}

```

```
(kali㉿kali)-[~/Desktop]
$ vi sample12.c

(kali㉿kali)-[~/Desktop]
$ gcc sample12.c -o sample12 -lm
```

Screenshot 2 – Choice 1 output

```
(kali㉿kali)-[~/Desktop]
$ ./sample12
1. Check increasing order
2. Check decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
Enter your choice: 1
Enter a positive integer: 12345
Digits are in increasing order
```

```
(kali㉿kali)-[~/Desktop]
$ ./sample12
1. Check increasing order
2. Check decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
Enter your choice: 1
Enter a positive integer: 12435
Digits are not in increasing order
```

Screenshot 3 – Choice 2 output

```
(kali㉿kali)-[~/Desktop]
$ ./sample12
1. Check increasing order
2. Check decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
Enter your choice: 2
Enter a positive integer: 97531
Digits are in decreasing order
```

```
(kali㉿kali)-[~/Desktop]
$ ./sample12
1. Check increasing order
2. Check decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
Enter your choice: 2
Enter a positive integer: 97351
Digits are not in decreasing order
```

Screenshot 4 – Choice 3 output

```
(kali㉿kali)-[~/Desktop]
$ ./sample12
1. Check increasing order
2. Check decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
Enter your choice: 3
Enter a positive integer: 58320
Largest digit = 8
Smallest digit = 0
```

Screenshot 5 – Choice 4 output

```
(kali㉿kali)-[~/Desktop]
$ ./sample12
1. Check increasing order
2. Check decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
Enter your choice: 4
Enter a positive integer: 58320
Second largest digit = 5
Second smallest digit = 2
```

```
(kali㉿kali)-[~/Desktop]
$ ./sample12
1. Check increasing order
2. Check decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
Enter your choice: 4
Enter a positive integer: 7777
Second largest digit does not exist
Second smallest digit does not exist
```